CS 578: CYBER-SECURITY

PART II: OS SECURITY I

Sanghyun Hong
sanghyun.hong@oregonstate.edu





ADDRESS SANITIZER – HOW TO DISCOVER BUGS BEFORE WE SHIP

BY TAK YEUNG AND AMISH

RECAP – WHERE WE ARE

- Part I – Network/Internet security
  - The attacker is *outside* your machine
- Part II – Computer systems security
  - The attacker is *inside* your machine
  - Memory safety: how the attacker gets inside your machine (Buffer overflow)
  - OS security: once inside, how can we *reduce* what the attacker can do?
    - SetUID demystified – drop the privileges that you don't need
    - Capsicum – Never hold those privileges in the first place

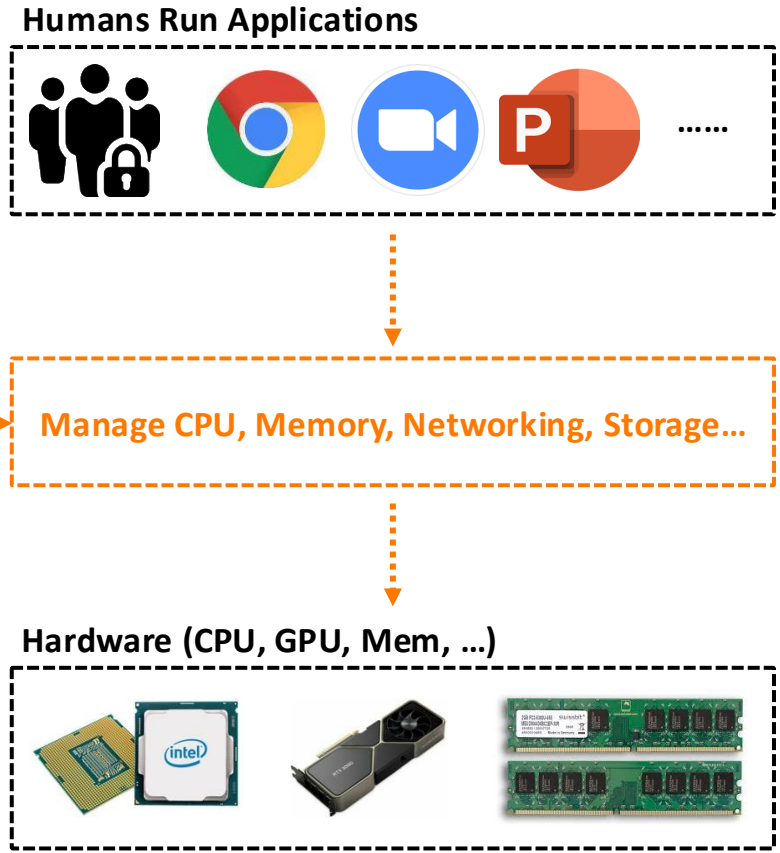
OPERATING SYSTEMS

- What is an operating systems?
  - Computer software that lies between hardware and applications



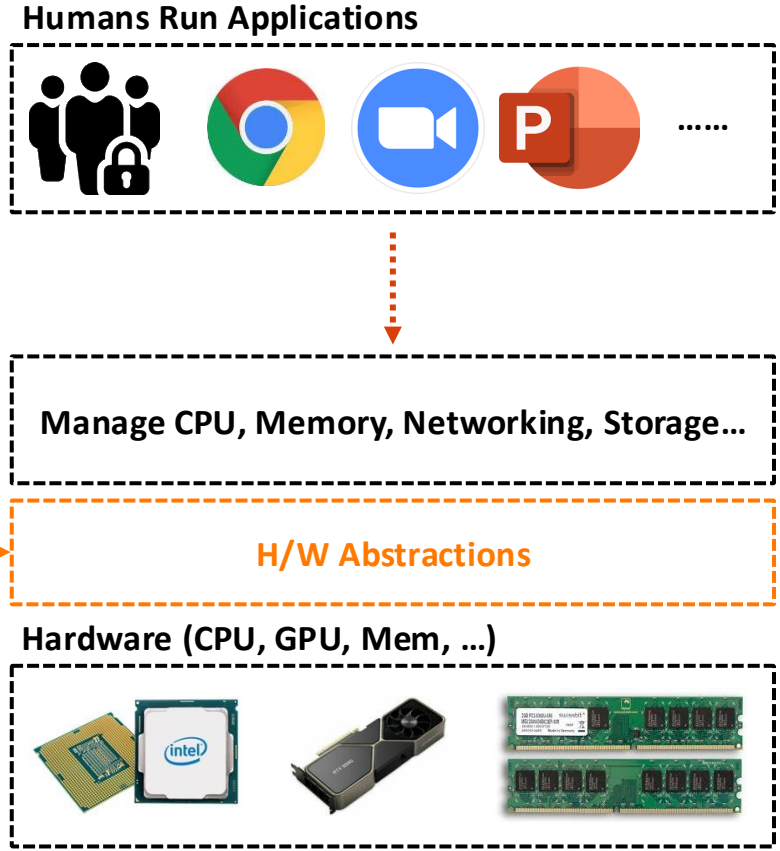
OPERATING SYSTEMS – CONT'D

- What does it do?
  - **Manage resources**
  - Provide abstractions
  - Offer standard interfaces



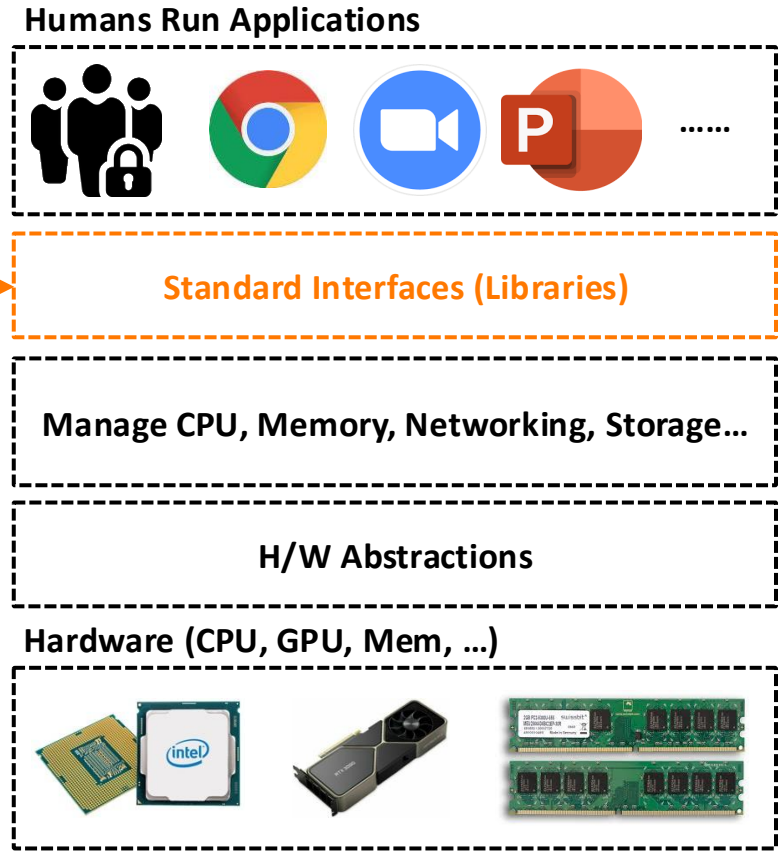
OPERATING SYSTEMS – CONT'D

- What does it do?
  - Manage resources
  - **Provide abstractions**
  - Offer standard interfaces



OPERATING SYSTEMS – CONT'D

- What does it do?
  - Manage resources
  - Provide abstractions
  - **Offer standard interfaces**



*UNIX ACCESS CONTROL PRELIM

LINUX ACCESS CONTROL

- Everything is a file
  - **Definition:** a named collection of data (*e.g.*, movie.csv containing movie data)
  - **POSIX : a sequence of data bytes**
  - ***NIX OS : everything**
    - Files on secondary storages, *e.g.*, disks
    - Devices (mouse, keyboard, monitor, …)
    - Network devices (network card, sockets in OS, …)
    - Inter-process communications (pipes, sockets, …)

LINUX ACCESS CONTROL – CONT'D

- Directories
  - **Definition :** a folder containing files and directories
  - **Motivation:**
    - Scenario: one day you create 100k+ files and the next day, you want to use them
  - **Solution :**
    - **S0:** You are Von Neumann; remember all the files
    - **S1:** Your system creates a folder containing all the files for each user
    - **S2:** Your system creates multiple folders containing the same kinds

LINUX ACCESS CONTROL – CONT'D

~/lecture/CS578$ ls -alh

total 312K

Permission	# hard-link	owner	owner-group	size (b)	last modified	name
drwxrwx---.	6	sahong	upg1xxxx	186	Apr 10 22:14	.
drwxrwx---.	3	sahong	upg1xxxx	73	Apr 5 19:58	..
drwxrwx---.	2	sahong	upg1xxxx	95	Apr 5 19:58	bufferoverflow
drwxrwx---.	2	sahong	upg1xxxx	52	Apr 4 09:02	bufferoverrun
drwxrwx---.	8	sahong	upg1xxxx	299	Apr 10 21:56	.git
-rw-rw----.	1	sahong	upg1xxxx	430	Apr 5 19:56	.gitignore
lrwxrwxrwx.	1	sahong	upg1xxxx	22	Apr 10 22:14	home -> /nfs/stak/users/hongsa
-rw-rw----.	1	sahong	upg1xxxx	44	Apr 4 08:15	README.md
drwxrwx---.	2	sahong	upg1xxxx	79	Apr 5 20:07	thread

owner | group | others X r | w | x

LINUX ACCESS CONTROL – CONT'D

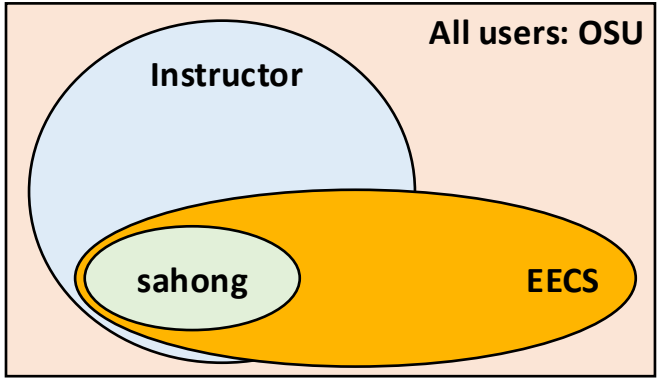
~/lecture/CS578$ ls -alh

```
total 312K
drwxrwx---.   6   sahong   upg1xxxx   186   Apr 10 22:14   .
drwxrwx---.   3   sahong   upg1xxxx   73    Apr  5 19:58   ..
drwxrwx---.   2   sahong   upg1xxxx   95    Apr  5 19:58   bufferoverflow

... <omit the entries>
```

Permission	# hard-link	owner	owner-group	size (b)	last modified	name

- Linux controls the access to files or directories based on three categories:
  - **u**ser : owner of a file or a directory
  - **g**roup : the group where users are
  - **o**thers: all the other users



LINUX ACCESS CONTROL – CONT'D

~/lecture/CS578$ ls -alh

total 312K

Permission	# hard-link	owner	owner-group	size (b)	last modified	name
		… <omit the entries>				
-rw-rw----.	1	sahong	upg1xxxx	44	Apr 4 08:15	README.md
drwxrwx---.	2	sahong	upg1xxxx	79	Apr 5 20:07	thread

- Permission representation
  - drwxrwx---

[Type] d: directory, -: file

[User] the first three letters (rwx)
[Group] the second three letters (rwx)
[Others] the last three letters (---)

LINUX ACCESS CONTROL – CONT'D

~/lecture/CS578$ ls -alh

total 312K

Permission	# hard-link	owner	owner-group	size (b)	last modified	name
	… <omit the entries>					
-rw-rw----.	1	sahong	upg1xxxx	44	Apr 4 08:15	README.md
drwxrwx---.	2	sahong	upg1xxxx	79	Apr 5 20:07	thread

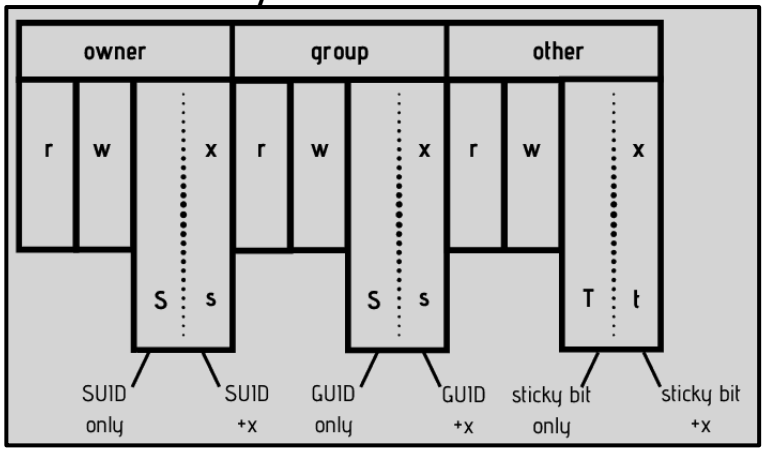
- Permission representation
  - drwxrwx---:
    - 770
    - 111111000

Interpretation

- Decimal #: 1st (user), 2nd (group), 3rd (others)
  + ex. 770 : 7 (user), 7 (group), 0 (others)
- Each #: Binary number
  1st (read), 2nd (write), 3rd (execute)
  + ex. 7 : 111 (rwx)
  + ex. 6 : 110 (rw)
  + ex. 600 : 110 000 000 (your ssh key)

LINUX ACCESS CONTROL – CONT'D

- Permission
  - **Read** : one can read files and directories with 'r' permission
  - **Write** : one can write files and dirs. with 'w' permission
  - **Execute:** one can execute files and dirs. with 'x' permission
  - **SetUID** : one can execute files and dirs. with the permissions of the owner/group of the command
  - **sticky** : except the creator and the root, no one can modify or delete the file



LINUX ACCESS CONTROL – CONT'D

```
~/lecture/CS578$ ls -alh /usr/bin/passwd
-rwsr-xr-x.  1  root root  32K  May 14  2022  /usr/bin/passwd
```

- An example /usr/bin/passwd
  - Users can update their authentication tokens with this program
  - Only an administrator with the root privilege should update the tokens
  - SetUID bit (s) enables this
    - Raise privilege only when needed
    - Drop it as soon as it becomes not-needed

LINUX ACCESS CONTROL – CONT'D

```
~/lecture/CS578$ ls -alh /usr/bin/passwd
-rwsr-xr-x.  1  root root  32K  May 14  2022  /usr/bin/passwd
```

- Buffer overflow in such binaries = root
  - How many do they exist? $ find / -perm -4000 -type f 2>/dev/null
  - How large does the attack surface?

SETUID DEMYSTIFIED – HOW TO DISCOVER BUGS BEFORE WE SHIP BY ZIMMER OSBORNE

CAPSICUM – COMPARTMENTALIZATION

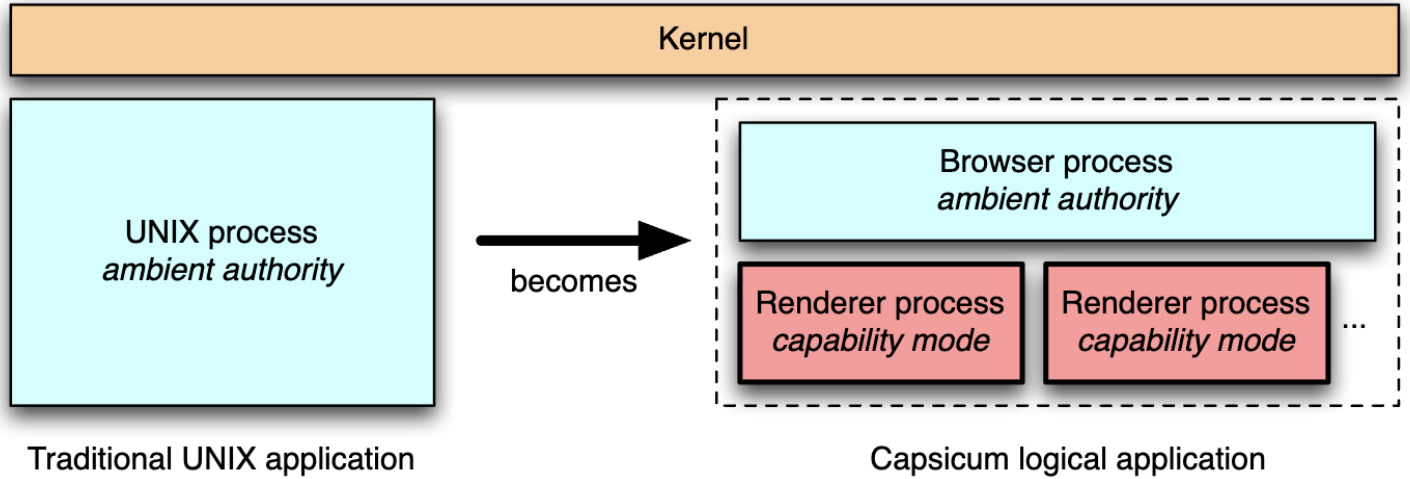
THE LIMITATION OF SETUID

- Suppose that you decompress a program with gzip
  - Gzip unfortunately contains a buffer overflow vulnerability
  - setuid makes sure that the attacker cannot access /etc/passwd
  - But…

- Your ~/.ssh/<ssh_key>
  - The attacker can read ~/.ssh/id_rsa and send it anywhere
  - The code runs with *your UID*
  - setuid protects your program from Alice
  - But it cannot from your own program processing untrusted input

CAPSICUM

- Key idea: compartmentalization



  - Split one program into pieces with different privileges
  - Privileged part (opens files, sockets) vs. Sandboxed part (parses the dangerous input)

SO FAR, OS IS NOT DESIGNED TO SUPPORT THIS

Chromium: one browser, six different sandbox implementations

OS	Mechanism	Lines of code
Windows	ACLs + SIDs	22,350
Linux	chroot (setuid-root helper)	605
Mac OS X	Seatbelt (MAC)	560
Linux	SELinux (MAC)	200
Linux	seccomp + userspace wrapper	11,301
FreeBSD	Capsicum	100

*Lines of code = how little the OS helped; not how safe it is

How to implement the compartmentalization

- Opening a file needs two steps:
  - Use a system call with names, e.g., open("/home/<usr>/.ssh/<your-key>")
  - OS then checks who asks – "Is this process's user allowed to do it?"

- Capsicum: no names, no "who," only the keys

BEFORE		AFTER GRANTED	
open("/etc/passwd")	O	open("/etc/passwd")	X
read(fd_input)	O	read(fd_input)	O

- Two kernel additions
  - cap_enter() do not use name-based approaches
  - Capabilities do use keys that grant what they can open

CAPSICUM – CAPABILITY MODE

- After cap_enter(), anything that needs a NAME fails
  - open("/etc/passwd") X file paths
  - kill(123, SIGKILL) X process IDs
  - connect("8.8.8.8:53") X network addresses

- Anything that a process holds keep working:
  - read(fd_input) O
  - write(fd_output) O

- One-way ticket
  - setuid can keep a way back (root is in the saved UID)
  - Capsicum has no way back at all; no system call to leave

CAPSICUM – CAPABILITIES

- Capabilities = file descriptors + rights
  - cap_new(fd, rights) create a key (fd) that can do only the “rights”
  - pdfork fork that hands the key to the child, not a PID

```
// ① BEFORE the lock — names still work. Grab what you will need
int lib = open("/lib", O_RDONLY | O_DIRECTORY);   // a DIRECTORY key
int fd  = open("photo.jpg", O_RDWR);              // read + write
int ro  = cap_new(fd, CAP_READ | CAP_SEEK);       // weaker copy    ✓

// ② LOCK
cap_enter();

// ③ AFTER the lock — names are gone, keys still work
open("/lib/libc.so.7", O_RDONLY);                 // a name         ✗
openat(lib, "libc.so.7", O_RDONLY);               // under /lib     ✓
openat(lib, "../etc/passwd", O_RDONLY);           // climbs out     ✗
read(ro, buf, n);                                 // has READ       ✓
write(ro, buf, n);                                // no WRITE       ✗
cap_new(ro, CAP_READ | CAP_WRITE);                // add a right?   ✗
```

CAPSICUM – TCPDUMP IN TWO LINES

```
+       if (cap_enter() < 0)
+               error("cap_enter: %s", pcap_strerror(errno));
        status = pcap_loop(pd, cnt, callback, pcap_userdata);
```

Figure 6: A two-line change adding capability mode to `tcpdump`: `cap_enter` is called prior to the main `libpcap` (packet capture) work loop. Access to global file system, IPC, and network namespaces is restricted.

- OPEN > LOCK > WORK
  - The historically buggy packet parser now runs with almost nothing

CAPSICUM – TCPDUMP IN TWO LINES

- But, “stdin” still readable (your keystrokes!)

```
+       if (lc_limitfd(STDIN_FILENO, CAP_FSTAT) < 0)
+               error("lc_limitfd: unable to limit STDIN_FILENO");
+       if (lc_limitfd(STDOUT_FILENO, CAP_FSTAT | CAP_SEEK | CAP_WRITE) < 0)
+               error("lc_limitfd: unable to limit STDOUT_FILENO");
+       if (lc_limitfd(STDERR_FILENO, CAP_FSTAT | CAP_SEEK | CAP_WRITE) < 0)
+               error("lc_limitfd: unable to limit STDERR_FILENO");
```

Figure 7: Using `lc_limitfd`, `tcpdump` can further narrow rights delegated by inherited file descriptors, such as limiting permitted operations on `STDIN` to `fstat`.

- stdin CAP_FSTAT can only read the file information
- stdout FSTAT | SEEK | WRITE can only print out
- stderr FSTAT | SEEK | WRITE can only print out

CAPSICUM – EXERCISE

- Let's sandbox an image decoder
  - What must it have before cap_enter()?
  - Which capabilities does each key need?
  - What might it try to open late …and break?

CAPSICUM – COST

- Entering the sandbox
  - cap_enter 1.22us
  - chroot 1.21us < setuid 1.39us
- Read on a capability
  - + 0.057us (+8.9%)
- A *new* sandbox process
  - libcapsicum: 1,509µs
  - vs. fork+exec 812µs (+86%)
  - Gzip: +2.4ms per run

Benchmark	Time/operation	Difference	% difference
dup	$1.061 \pm 0.000\mu s$	-	-
cap_new	$1.600 \pm 0.001\mu s$	$0.539 \pm 0.001\mu s$	$50.7\% \pm 0.08\%$
shmfd	$2.385 \pm 0.000\mu s$	-	-
cap_new_shmfd	$4.159 \pm 0.007\mu s$	$1.77 \pm 0.004\mu s$	$74.4\% \pm 0.181\%$
fstat_shmfd	$0.532 \pm 0.001\mu s$	-	-
fstat_cap_shmfd	$0.586 \pm 0.004\mu s$	$0.054 \pm 0.003\mu s$	$10.2\% \pm 0.506\%$
read_1	$0.640 \pm 0.000\mu s$	-	-
cap_read_1	$0.697 \pm 0.001\mu s$	$0.057 \pm 0.001\mu s$	$8.93\% \pm 0.143\%$
read_10000	$1.534 \pm 0.000\mu s$	-	-
cap_read_10000	$1.601 \pm 0.003\mu s$	$0.067 \pm 0.002\mu s$	$4.40\% \pm 0.139\%$
write	$0.576 \pm 0.000\mu s$	-	-
cap_write	$0.634 \pm 0.002\mu s$	$0.058 \pm 0.001\mu s$	$10.0\% \pm 0.241\%$
cap_enter	$1.220 \pm 0.000\mu s$	-	-
getuid	$0.353 \pm 0.001\mu s$	$-0.867 \pm 0.001\mu s$	$-71.0\% \pm 0.067\%$
chroot	$1.214 \pm 0.000\mu s$	$-0.006 \pm 0.000\mu s$	$-0.458\% \pm 0.023\%$
setuid	$1.390 \pm 0.001\mu s$	$0.170 \pm 0.001\mu s$	$14.0\% \pm 0.054\%$
fork	$268.934 \pm 0.319\mu s$	-	-
vfork	$44.548 \pm 0.067\mu s$	$-224.3 \pm 0.217\mu s$	$-83.4\% \pm 0.081\%$
pdfork	$259.359 \pm 0.118\mu s$	$-9.58 \pm 0.324\mu s$	$-3.56\% \pm 0.120\%$
pingpong	$309.387 \pm 1.588\mu s$	$40.5 \pm 1.08\mu s$	$15.0\% \pm 0.400\%$
fork_exec	$811.993 \pm 2.849\mu s$	-	-
vfork_exec	$585.830 \pm 1.635\mu s$	$-226.2 \pm 2.183\mu s$	$-27.9\% \pm 0.269\%$
pdfork_exec	$862.823 \pm 0.554\mu s$	$50.8 \pm 2.83\mu s$	$6.26\% \pm 0.348\%$
sandbox	$1509.258 \pm 3.016\mu s$	$697.3 \pm 2.78\mu s$	$85.9\% \pm 0.339\%$

RECAP

	UNIX / setuid	Capsicum
Authority	from WHO you are (uid)	WHAT you hold (fd + rights)
To drop it	change identity: 3 uids 4 syscalls, OS-dependent	close or narrow a handle
Default	everything your user can do	nothing — after cap_enter
Policy	lives in file permission bits	in the program's own code

Thank You!

Sanghyun Hong

https://true-lab.ai/courses/Sec-Grad/current



